Efficient first neighbor analysis in large atomic scale models [Article v1.0]

Sébastien Le Roux[1*†]

[1]Institut de Physique et Chimie des Matériaux de Strasbourg, Université de Strasbourg, CNRS UMR 7504, 23 rue de Loess, BP 43, F-67034 Strasbourg Cedex 2, France

This LiveCoMS document is maintained online on GitHub at https://github.com/Slookeur/Bonds; to provide feedback, suggestions, or help improve it, please visit the GitHub repository and participate via the issue tracker.

This version dated June 29, 2026

Abstract This article describes how lattice mathematical properties can be combined with the 3D space pixelation method to evaluate interatomic distances in large atomic scale 3D models with the purpose of searching for first neighbor atoms. It aims to be an educational tool for students and researchers who want to develop structural analysis or 3D visualization tools that requires to implement and compute efficiently the search for first neighbor atoms.
Examples codes are provided in C, FORTRAN90 and Python.

***For correspondence:**
sebastien.leroux@ipcms.unistra.fr (SLR)

[†]This author is the sole author of this work

1 Introduction

Every student, every researcher in computational material science, has already spent time calculating interatomic distances. This problem is even likely to be the first one computational material scientists will spend some time over during their studies. As simple as the evaluation of a distance in 3D space could seem to be, the complexity of the problem increases considerably when dealing with the periodicity of non-cubic systems, and even more with the search for performance that is driving the analysis and the visualization of atomic scale models with more than tens of thousands of atoms.

This manuscript illustrates how lattice mathematical properties can be combined with the pixelation of the model box approach, to offer both a general, symmetry independent, methodology, and, an extremely efficient implementation of the search for first neighbor atoms.

2 Simulation box, lattice parameters and transformation matrices

Knowledge and understanding of the mathematics of lattice parameters and atomic coordinates is a prerequisite to the general formulation of the calculation of interatomic bond distances in 3D atomic scale models using periodic boundary conditions.
Note that this section can safely be ignored when dealing with non periodic systems.

2.1 Simulation box or lattice parameters

Box, or lattice parameters can be expressed with two different sets of parameters, using:

1. Using the box parameters A, B, C and the associated angles α, β and γ
2. Using the components of the lattice vectors $\vec{a}(a_x, a_y, a_z)$, $\vec{b}(b_x, b_y, b_z)$ and $\vec{c}(c_x, c_y, c_z)$

Then lattice vectors (2.1) can be calculated using box parameters (2.1) with:

$$\begin{bmatrix} A & 0.0 & 0.0 \\ \times\cos\gamma & B\times\sin\gamma & 0.0 \\ C\times\cos\beta & C\times L & C\times L^2 \end{bmatrix} \quad (1)$$

With:

$$L = \frac{\cos\alpha - \cos\beta\times\cos\gamma}{\sin\gamma} \quad (2)$$

While box parameters (2.1) can be calculated using lattice vectors (2.1) with:

$$A = |\,\vec{a}\,| \quad B = |\,\vec{b}\,| \quad C = |\,\vec{c}\,| \quad (3)$$

and:

$$\alpha = \frac{\vec{c}\cdot\vec{b}}{B\times C} \quad \beta = \frac{\vec{a}\cdot\vec{c}}{A\times C} \quad \gamma = \frac{\vec{a}\cdot\vec{b}}{A\times B} \quad (4)$$

The lattice volume:

$$V = \vec{a}\cdot(\vec{b}\wedge\vec{c}) = \vec{b}\cdot(\vec{c}\wedge\vec{a}) = \vec{c}\cdot(\vec{a}\wedge\vec{b}) \quad (5)$$

can then be calculated using:

$$V = A\times B\times C\times \mathbf{Z} \quad (6)$$

With:

$$\mathbf{Z} = \sqrt{1 - \cos^2\alpha - \cos^2\beta - \cos^2\gamma + 2\,\cos\alpha\,\cos\beta\,\cos\gamma} \quad (7)$$

Knowledge of these properties is a basic requirement, from there it possible to compute transformation matrices that allow the conversion from Cartesian *r* to Fractional *f* coordinates and the conversion from Fractional to Cartesian coordinates. These mathematical tools are extremely useful, if not almost mandatory prerequisites to the calculation, when dealing with non-cubic periodic systems.

2.2 From Cartesian to fractional coordinates

For an atom with Cartesian coordinates (r_x,r_y,r_z), fractional coordinates (f_x,f_y,f_z) can be calculated using:

$$\begin{pmatrix} f_x \\ f_y \\ f_z \end{pmatrix} = \mathbf{T_f} \times \begin{pmatrix} r_x \\ r_y \\ r_z \end{pmatrix} \quad (8)$$

Where the transformation matrix $\mathbf{T_f}$ is defined as:

$$\mathbf{T_f} = \begin{bmatrix} \frac{1}{A} & -\frac{\cos\gamma}{A\sin\gamma} & \frac{\cos\alpha\cos\gamma-\cos\beta}{A\mathbf{Z}\sin\gamma} \\ 0.0 & \frac{1}{B\sin\gamma} & \frac{\cos\alpha\cos\gamma-\cos\beta}{B\mathbf{Z}} \\ 0.0 & 0.0 & \frac{\sin\gamma}{C\mathbf{Z}} \end{bmatrix} \quad (9)$$

2.3 From fractional to Cartesian coordinates

Similarly fractional coordinates (f_x,f_y,f_z) can be converted to Cartesian coordinates (r_x,r_y,r_z) using:

$$\begin{pmatrix} r_x \\ r_y \\ r_z \end{pmatrix} = \mathbf{T_c} \times \begin{pmatrix} f_x \\ f_y \\ f_z \end{pmatrix} \quad (10)$$

Where the transformation matrix $\mathbf{T_c}$ is defined as:

$$\mathbf{T_c} = \mathbf{T_f}^{-1} = \begin{bmatrix} A & B\cos\gamma & C\cos\beta \\ 0.0 & B\sin\gamma & C\,\frac{\cos\alpha-\cos\beta\cos\gamma}{\sin\gamma} \\ 0.0 & 0.0 & \frac{C\mathbf{Z}}{\sin\gamma} \end{bmatrix} \quad (11)$$

3 Pixelation of the model box

The idea of pixelation, or partitioning, of the model box illustrated in this section is mandatory to deal efficiently with searching for neighbor atoms in large atomic scale models. Indeed the intuitive way to implement the procedure would be to test every pair $i-j$ of atoms in the model: compute the interatomic distance D_{ij} between i (α) and j (β), and then compared this distance to a cutoff radius $R_{cut}(\alpha,\beta)$, that could appropriately be determined when looking at the radial distribution function $g_{\alpha\beta}(r)$. Then if D_{ij} is smaller or equal to $R_{cut}(\alpha,\beta)$ then atoms i and j are first neighbors, otherwise they are not. For a program which purpose is to render the atomic scale model in 3D space, the result of the analysis would be then to draw, or not, a bond between atoms i and j.

As intuitive and logical as this approach could seem to be, it requires to perform the testing for every pair of atoms in the model. Which, as long as the size of the system remains within the thousand or few thousands of atoms, could work in a seemingly efficient manner. The time order for the entire analysis is then proportional to $\frac{N\times(N-1)}{2}$, with N the total number of atoms in the model. However as the number of atoms increases, time required to performed the entire analysis increases even more dramatically, soon enough reaching a point where the program will likely seem to be completely frozen.

Therefore another approach is required to optimize the procedure for large atomic scale models, and that is to distinguish atom(s) that are of interest for the purpose of the calculation from atom(s) that are not. This is done by dividing, or partitioning, the model box in smaller parts, or pixels.

Atomic coordinates will allow to associate an atom to a particular pixel. Then for this particular atom neighbors candidates will only be search for in that same pixel and its

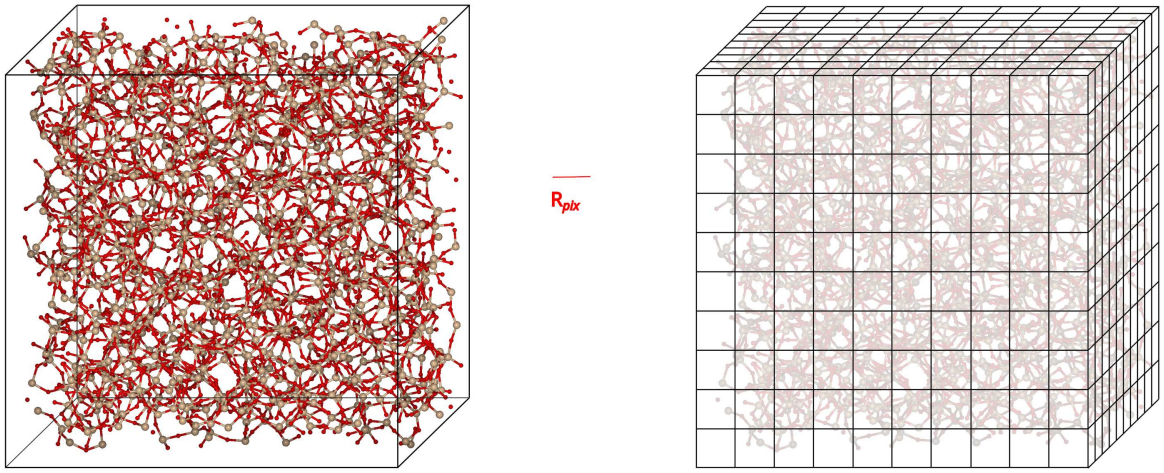


Figure 1. Pixelation of the model box

immediate surrounding pixel neighbors. Ideally the pixel dimension R_{pix} should be close but slightly superior to R_{cut} the cutoff radius used to search for first neighbors atoms, ex: $R_{pix} = R_{cut} + 0.5$.

For any atom in the model this approach will limit the area of interest to the smallest possible size. Using this methodology the time order for the entire analysis becomes proportional to $N_p \times \frac{N_c \times (N_c - 1)}{2}$, with N_p is the total number of pixels in the grid, and N_c is the average number of atom(s) in the pixel and its 26 surrounding neighbor pixels: $N_c <<< N_p << N$.
The idea behind this approach is illustrated in figure 1.

3.1 Without Periodic Boundary Conditions

The number of pixels on each axis, $n_p(x)$, $n_p(y)$ and $n_p(z)$, are calculated using:

$$n_p(axis) = \left\lceil \frac{D_{max}(axis)}{R_{pix}} \right\rceil \tag{12}$$

Where $D_{max}(axis)$ is the maximum interatomic distance separating two atoms on *axis*, and $R_{pix} \simeq R_{cut}$ is the pixel dimension, close to the cutoff distance that separates neighbor atoms.
Providing a model box, with parameters A, B, and C, encompassing the entire model could prove useful here, allowing the simplifications:

$$D_{max}(x) = A, \qquad D_{max}(y) = B \qquad \text{and} \qquad D_{max}(z) = C \tag{13}$$

Otherwise calculations to determine D_{max} for each axis are needed. It is then required to test each pair of atomic coordinates in the model on x, y and z. However as long as only subtractions and min/max comparisons are involved calculation time will remain acceptable.

Then the total number of pixels in the model box, *pixels*, is calculated using:

$$pixels = n_p(x) \times n_p(y) \times n_p(z) \tag{14}$$

For an atom *at* with Cartesian coordinates (r_x, r_y, r_z) in the model, corresponding pixel indices in the pixel grid (p_x, p_y, p_z) can be calculated using:

$$p_{axis} = \left\lfloor \frac{r_{axis} - min_{axis}}{R_{pix}} \right\rfloor \tag{15}$$

With:

$$p_{axis} \in [0, n_p(axis) - 1] \tag{16}$$

Where min_{axis} in the lowest value for any atomic coordinates in the model on *axis*.
The pixel number for *at*, between 0 and $pixels - 1$, in entire the pixel grid, P_{id} is calculated using:

$$P_{id}(at) = p_x + n_p(x) \times p_y + \left[n_p(x) \times n_p(y)\right] \times p_z \tag{17}$$

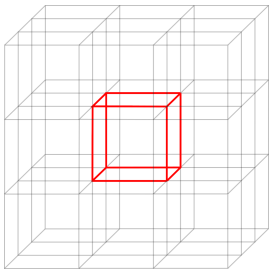
With:

$$P_{id} \in [0, pixels - 1] \tag{18}$$

First neighbors list is calculated as follow:

1. The pixel position (p_x, p_y, p_z) for every atom (r_x, r_y, r_z) in the model is to be calculated so that each atom can be assigned a pixel number in the grid.
2. Accordingly a list of atom containing pixels is created, the list of atom(s) in each pixel being stored.

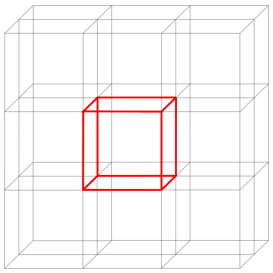
3. First neighbor(s) for an atom will then be search for in the pixel this atom belong to and in its surrounding pixel neighbors only, all other pixel(s) being safely ignored.
   The list of pixel neighbors is constructed using the mathematical relationships between each pixel index in the grid, two cases must considered:
   - Pixel inside the pixel grid:



Self + 26 neighbors

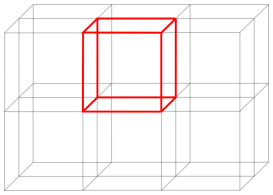
   - Pixel on the boundary of the pixel grid : $p_{axis} = 0$ or $p_{axis} = n_p(axis) - 1$

Face of the pixel grid



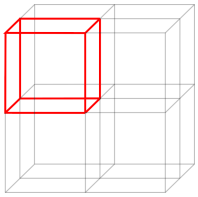
Self + 17 neighbors

Edge of the pixel grid



Self + 11 neighbors

Corner of the pixel grid



Self + 7 neighbors

   Pixel neighbors are determined as illustrated in figure 2:

Note that this kind of approach only makes sense if the number of pixels in the grid is high enough so that all pixels are not neighbors.

3.2 With Periodic Boundary Conditions

Non-cubic symmetries make it more complicated to offer a general methodology to deal with periodic systems:

1. Evaluate the number of pixel(s) on each axis, $n_p(x)$, $n_p(y)$ and $n_p(z)$, using equations 12 and 13.
2. Convert atomic Cartesian coordinates to fractional coordinates using $\mathbf{T_f}$.
   Using fractional coordinates is the easiest way to propose a general methodology to compute interatomic distances in the model. Indeed this requires to consider the periodicity of the system, and, in the case of non-cubic symmetry transformations, could be tricky when using Cartesian coordinates. Working with fractional coordinates is much easier since in that case corrections are performed simply adding or subtracting multiples of 1.0 on any fractional direction.
   - Convert Cartesian coordinates to fractional coordinates (f_x, f_y, f_z) using Eq. 8.
   - Compute corrected fractional coordinates ($f_{c,x}$, $f_{c,y}$, $f_{c,z}$) inside the model box:

$$f_{c,axis} = f_{axis} - \lfloor f_{axis} \rfloor \tag{19}$$

   With:

$$0 \leq f_{c,axis} < 1 \tag{20}$$

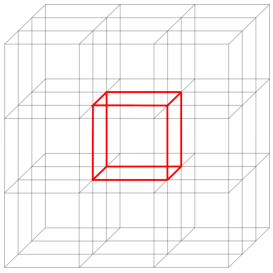
3. Pixel positions (p_x, p_y, p_z) are determined using the atom's corrected fractional coordinates ($f_{c,x}$, $f_{c,y}$, $f_{c,z}$):

$$p_{axis} = \lfloor f_{c,axis} \times n_p(axis) \rfloor \tag{21}$$

   With:

$$p_{axis} \in [0, n_p(axis) - 1] \tag{22}$$

4. Determine each pixel neighbors:
   - Pixel inside the pixel grid:



Self + 26 neighbors, no PBC transformation required.

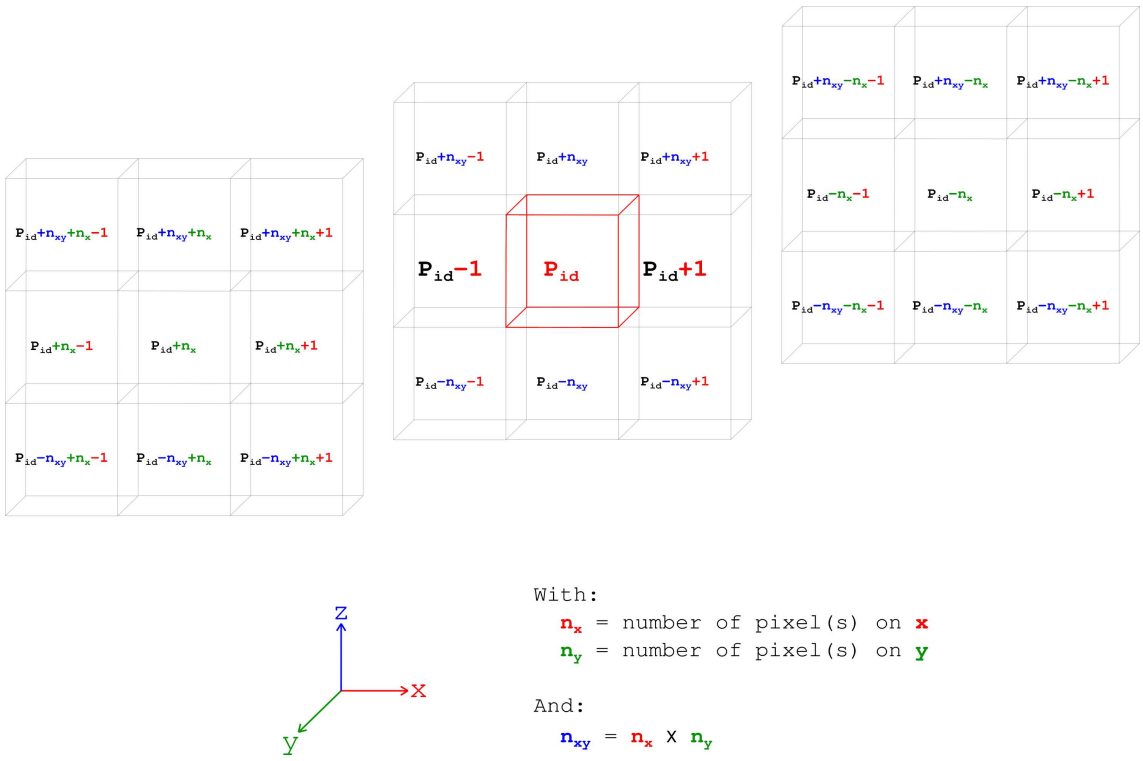
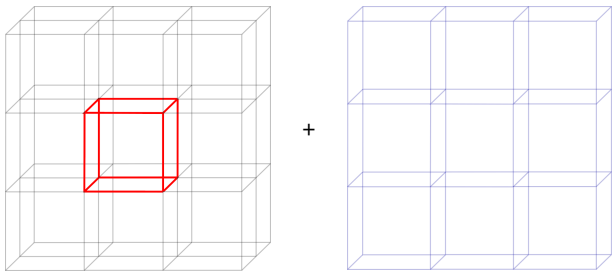


Figure 2. Finding pixel neighbors for pixel P_{id}: operation(s) on each axis are illustrated in the appropriate color(s)

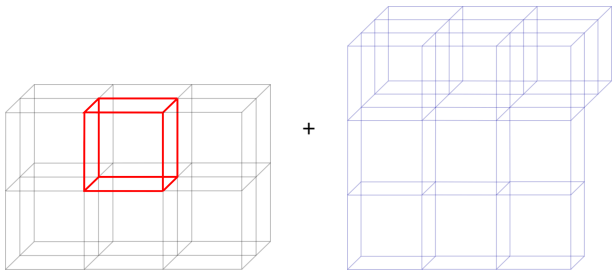
- Pixel on the boundary of the pixel grid :
  $p_{axis} = 0$ or $p_{axis} = n_p(axis) - 1$

Face of the pixel grid:



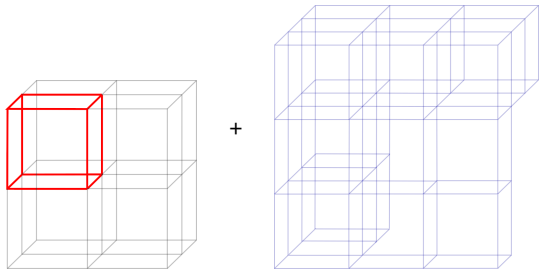
Self + 17 + 9 neighbors using PBC

Edge of the pixel grid:



Self + 11 + 15 neighbors using PBC

Corner of the pixel grid:



Self + 7 + 19 neighbors using PBC

For a pixel with number p_{id} with pixel coordinates (p_x, p_y, p_z) inside the pixel grid, pixel neighbors are determined as illustrated in figure 2. For a pixel on the boundary of the grid, as the 3 cases described above, that means if $p_{axis} = 0$ or $p_{axis} = n_p(axis) - 1$, then it is possible to determine its PBC pixel neighbors by applying a shift to the pixel neighbors number calculated as illustrated in fig. 2.

In the in the 3 directions of space:

if $p_x =$	0	shift all	$p_x - 1$	pixels by	$n_p(x)$
if $p_x =$	$n_p(x) - 1$	shift all	$p_x + 1$	pixels by	$-n_p(x)$
if $p_y =$	0	shift all	$p_y - 1$	pixels by	$n_p(xy)$
if $p_y =$	$n_p(y) - 1$	shift all	$p_y + 1$	pixels by	$-n_p(xy)$
if $p_z =$	0	shift all	$p_z - 1$	pixels by	$n_p(xyz)$
if $p_z =$	$n_p(z) - 1$	shift all	$p_z + 1$	pixels by	$-n_p(xyz)$

With:

$$n_p(xy) = n_p(x) \times n_p(y) \tag{23}$$

And:

$$n_p(xyz) = n_p(xy) \times n_p(z) \tag{24}$$

5. When searching for first neighbor atoms between atoms *i* and *j* of the same pixel or one of its surrounding pixel neighbor, compute the interatomic distance D_{ij} between *i* and *j* using their corrected fractional coordinates:

$$f_{ij}(axis) = f_{c,axis}(i) - f_{c,axis}(j) \tag{25}$$

$$f_{c,ij}(axis) = f_{ij}(axis) - \lfloor f_{ij}(axis) \rceil \tag{26}$$

$$\vec{r_{ij}} = \mathbf{T_c} \times \vec{f_{c,ij}} \tag{27}$$

$$\text{with} \quad \vec{r_{ij}} = \begin{pmatrix} r_{ij}(x) \\ r_{ij}(y) \\ r_{ij}(z) \end{pmatrix} \tag{28}$$

$$\text{and} \quad \vec{f_{c,ij}} = \begin{pmatrix} f_{c,ij}(x) \\ f_{c,ij}(y) \\ f_{c,ij}(z) \end{pmatrix} \tag{29}$$

$$D_{ij} = |\vec{r_{ij}}| \tag{30}$$

Note that some further calculation time can be spared if D_{ij}^2 only is calculated so that no squared root calculation is performed to obtain D_{ij}. Therefore in that case D_{ij}^2 must be compared to R_{cut}^2.
As mentioned in the previous section this approach only makes sense when the number of pixels in the grid is high enough so that all pixels are not neighbors. In the case where PBC are applied this means that the number of pixels on one dimension, *x*, *y*, or *z* should be higher than 3.

4 Limitations

Overall the pixelation, or partitioning, of the model box approach is extremely efficient. However some issues might arise when dealing with a very large system containing very few atoms, which could happen if:

- No periodicity is involved but the model is very large and describes a single, or few, isolated molecule(s) that are far apart.
- The description of the model relies on a very large box, usually with a simple cubic periodicity, that contains a single or few isolated molecule.

In such rare cases the number density, in atoms / Å^3, can be very low (< 0.01 atom / Å^3), but the number of pixels required to build the entire pixel grid can be too high for the memory allocation(s) required to perform the analysis. To overcome this issue the size of the pixel R_{pix} can be increased, to a value significantly higher than the cutoff radius R_{cut}, so that the density of atom(s) per pixel can be adjusted to match a proper target value to ensure that memory allocation will not be an issue, and that the analysis will be successful. By experience a value between 1.5 and 2 atoms per pixel is a safe window to avoid any issue in these peculiar cases.

5 Example codes

Commented codes that illustrate the entire work procedure are provided in the manuscript appendixes:

- C codes: A.1 through A.8.
- FORTRAN90 codes: B.1 through B.8.
- Python codes: C.1 through C.8.
  Note that this Python code is provided to illustrate the entire implementation in Python, but for that particular programming language several Python libraries already exist and can be used as fronted to simplify the complete coding (ex: ASE, Pysic, MDAnalysis), however in that case the pixelation approach is rarely coded in pure Python language (Pysic using FORTRAN, MDAnalysis using C).

6 Further optimizations

The analysis time can be reduced using MPI and/or OpenMP parallel programming. Several scenario, or approaches, can be envisioned depending on the size of the system in number of atoms and/or the number of configuration (MD steps):

- Single (MPI or OpenMP): atomic coordinates or pixels can be distributed over the CPU and/or CPU cores.
- Multiples (MPI or OpenMP): configurations can be distributed over the CPU and/or the CPU cores.
- Multiples (MPI and OpenMP): with hybrid parallelization configurations can be distributed over the CPU, and atomic coordinates or pixels can be distributed over the CPU cores.

Ideally the code would provide the option to switch between one and the other approach based on the number of configurations and or atoms in the system.
Note that this is the case of the **atomes** software [1] that implements an adaptive OpenMP programming distributing either the atomic coordinates or the MD steps on the CPU cores.

7 Conclusion

The general methodology behind efficient first neighbor(s) analysis in any kind of atomic scale models was described.
It was illustrated that the understanding of lattice mathematics can simplify the evaluation of interatomic bond distances independently of the periodicity of the system, and that the particular idea of the pixelation, or partitioning, of the model box, is a prerequisite to any modern implementation of this analysis.
The methodology described in this article is the one implemented in the **atomes** software [1].

Checklist

For a more detailed description of author contributions, see the GitHub issue tracking and changelog at https://github.com/Slookeur/Bonds.

Potentially Conflicting Interests

No conflicting interests.

Author Information

ORCID:
Sébastien Le Roux: 0000-0002-1912-6960

References

[1] **Le Roux S.** Comp. Mat. Sci.. 2025; 253:113805. https://doi.org/10.1016/j.commatsci.2025.113805.

SEARCHING FOR FIRST NEIGHBOR ATOMS

Non-periodic systems

- ☐ Choose a cutoff radius R_{cut} to determine first neighbor atoms, and the associated pixel dimension R_{pix}
- ☐ Find the maximum distances, $D_{max}(x)$, $D_{max}(y)$ and $D_{max}(z)$ that separate 2 atoms on x, y and z
- ☐ Determine the number of pixels, $n_p(axis)$, on x, y and z using $n_p(axis) = \left\lceil \frac{D_{max}(axis)}{R_{pix}} \right\rceil$
- ☐ Create a pixel grid for a virtual box with dimensions $n_p(x)$, $n_p(y)$ and $n_p(z)$ on x, y and z axis respectively.
- ☐ Using its Cartesian coordinates, $r(at)$, associate each atom at with a pixel position in the grid
  - ☐ Pixel position on each $axis$: $p_{axis} = \left\lfloor \frac{r_{at}(axis) - min_{axis}}{R_{pix}} \right\rfloor$
  - ☐ Pixel ID number in the grid: $P_{id}(at) = p_x + n_p(x) \times p_y + \left[n_p(x) \times n_p(y)\right] \times p_z$
- ☐ Associate each pixel with it surrounding neighbors in the grid
- ☐ For all pixels in the grid
  - ☐ Search for neighbors between the atoms of pixel pix as well as the atoms of its surrounding pixel neighbors pjx
    - ☐ For 2 atoms i in pixel pix and j in pixel pjx
      - ☐ Evaluate r_{ij} on x, y and z using atoms i and j Cartesian coordinates
      - ☐ Compute and return D_{ij}^2
      - ☐ Compare D_{ij}^2 to R_{cut}^2 to determine if i and j are first neighbors

Periodic systems

- ☐ Choose a cutoff radius R_{cut} to determine first neighbor atoms, and the associated pixel dimension R_{pix}
- ☐ Use the lattice parameters as maximum interatomic distances on x, y and z: $D_{max}(x) = A$, $D_{max}(y) = B$ and $D_{max}(z) = C$
- ☐ Determine the number of pixels, $n_p(axis)$, on x, y and z using $n_p(axis) = \left\lceil \frac{D_{max}(axis)}{R_{cut}} \right\rceil$
- ☐ Create a pixel grid with dimensions $n_p(x)$, $n_p(y)$ and $n_p(z)$ on x, y and z axis respectively.
- ☐ Convert all atom Cartesian coordinates to fractional coordinates f using $\mathbf{T_f}$
- ☐ Compute the corrected fractional coordinates f_c to ensure that all atoms are in the unit cell $f_c = f - \lfloor f \rfloor$
- ☐ Using its correct fractional coordinates, $f_c(at)$, associate each atom at with a pixel position in the grid
  - ☐ Pixel position on each $axis$: $p_{axis} = \left\lfloor f_{c,axis}(at) \times n_p(axis) \right\rfloor$
  - ☐ Pixel ID number in the grid: $P_{id}(at) = p_x + n_p(x) \times p_y + \left[n_p(x) \times n_p(y)\right] \times p_z$
- ☐ Associate each pixel with it surrounding neighbors in the grid, if needed apply PBC transformation(s)
- ☐ For all pixels in the grid
  - ☐ Search for neighbors between the atoms of pixel pix as well as the atoms of its surrounding pixel neighbors pjx
    - ☐ For 2 atoms i in pixel pix and j in pixel pjx
      - ☐ Evaluate f_{ij} on x, y and z using the atom i and j corrected fractional coordinates f_c
      - ☐ Correct f_{ij} to $f_{c,ij}$ for potential PBC correction(s) : $f_{c,ij} = f_{ij} - \lfloor f_{ij} \rceil$
      - ☐ Transform back $f_{c,ij}$ to Cartesian coordinates r_{ij} using $\mathbf{T_c}$
      - ☐ Compute and return D_{ij}^2
      - ☐ Compare D_{ij}^2 to R_{cut}^2 to determine if i and j are first neighbors

Appendix A Commented C code

A.1 C code: data structures and global variables

```c
// global data structures and variables used in the next code sections
#include <math.h>

#define TRUE 1
#define FALSE 0

// atom in pixel data structure
typedef struct pixel_atom pixel_atom;
struct pixel_atom
{
  int atom_id;             // the atom ID
  float coord[3];          // the atom coordinates on x, y and z
};

// pixel data structure
typedef struct pixel pixel;
struct pixel
{
  int pid;                 // the pixel number
  int p_xyz[3];            // the pixel coordinates in the grid
  bool tested;             // was the pixel checked already
  int patoms;              // number of atom(s) in pixel
  pixel_atom * pix_atoms;  // list of atom(s) in the pixel, to be allocated
  int neighbors;           // number of neighbors for pixel
  int pixel_neighbors[27]; // the list of neighbor pixels, maximum 27
};

// pixel grid data structure
typedef struct pixel_grid pixel_grid;
struct pixel_grid
{
  int pixels;              // total number of pixels in the grid
  int n_pix[3];            // number of pixel(s) on each axis
  int n_xy;                // number of pixels in the plan xy
  pixel * pixel_list;      // pointer to the pixels, to be allocated
};

// bond distance data structure
typedef struct distance distance;
struct distance
{
  float length;            // the distance in \AA\ squared
  float Rij[3];            // vector components of x, y and z
};

/*
  the following are considered to be provided by the user
*/

// model description
int atoms;                 // the total number of atom(s)
float ** c_coord;          // list of Cartesian coordinates: c_coord[atoms][3]
float cutoff;              // the cutoff to define atomic bond(s)
float cutoff_squared;      // squared value for the cutoff

// model box description, if PBC are applied
float volume;              // the lattice volume
float l_params[3];         // lattice a, b and c
float cart_to_frac[3][3];  // Cartesian to fractional coordinates matrix
float frac_to_cart[3][3];  // fractional to Cartesian coordinates matrix
```

A.2 C code: set periodic boundary condition pixel shift

```c
void set_pbc_shift (pixel_grid * grid, int pixel_coord[3], int pbc_shift[3][3][3])
{
  int x_pos, y_pos, z_pos; // loop iterators
  for ( x_pos = 0 ; x_pos < 3 ; x_pos ++ )
  {
    for ( y_pos = 0 ; y_pos < 3 ; y_pos ++ )
    {
      for ( z_pos = 0 ; z_pos < 3 ; z_pos ++ )
      {
        pbc_shift[x_pos][y_pos][z_pos] = 0;             // initialization without any shift
      }
    }
  }
  if ( pixel_coord[0] == 0 )                            // pixel position on 'x' is min
  {
    for ( y_pos = 0 ; y_pos < 3 ; y_pos ++ )
    {
      for ( z_pos = 0 ; z_pos < 3 ; z_pos ++ )
      {
        pbc_shift[0][y_pos][z_pos] = grid->n_pix[0];
      }
    }
  }
  else if ( pixel_coord[0] == grid->n_pix[0] - 1 )      // pixel position on 'x' is max
  {
    for ( y_pos = 0 ; y_pos < 3 ; y_pos ++ )
    {
      for ( z_pos = 0 ; z_pos < 3 ; z_pos ++ )
      {
        pbc_shift[2][y_pos][z_pos] = - grid->n_pix[0];
      }
    }
  }
  if ( pixel_coord[1] == 0 )                            // pixel position on 'y' is min
  {
    for ( x_pos = 0 ; x_pos < 3 ; x_pos ++ )
    {
      for ( z_pos = 0 ; z_pos < 3 ; z_pos ++ )
      {
        pbc_shift[x_pos][0][z_pos] += grid->n_xy;
      }
    }
  }
  else if ( pixel_coord[1] == grid->n_pix[1] - 1 )      // pixel position on 'y' is max
  {
    for ( x_pos = 0 ; x_pos < 3 ; x_pos ++ )
    {
      for ( z_pos = 0 ; z_pos < 3 ; z_pos ++ )
      {
        pbc_shift[x_pos][2][z_pos] -= grid->n_xy;
      }
    }
  }
  if ( pixel_coord[2] == 0 )                            // pixel position on 'z' is min
  {
    for ( x_pos = 0 ; x_pos < 3 ; x_pos ++ )
    {
      for ( y_pos = 0 ; y_pos < 3 ; y_pos ++ )
      {
        pbc_shift[x_pos][y_pos][0] += grid->pixels;
      }
    }
  }
  else if ( pixel_coord[2] == grid->n_pix[2] - 1 )      // pixel position on 'z' is max
  {
    for ( x_pos = 0 ; x_pos < 3 ; x_pos ++ )
    {
      for ( y_pos = 0 ; y_pos < 3 ; y_pos ++ )
      {
        pbc_shift[x_pos][y_pos][2] -= grid->pixels;
      }
    }
  }
}
```

A.3 C code: add atom to pixel

```c
void add_atom_to_pixel (pixel * the_pixel, int pixel_coord[3], int atom_id, float atom_coord[3])
{
  int axis;
  if (! the_pixel->patoms)
  {
    // if the pixel do not contains any atom yet, then save its coordinates in the grid
    for ( axis = 0 ; axis < 3 ; axis ++ )
    {
      the_pixel->p_xyz[axis] = pixel_coord[axis];
    }
    // allocate the memory to store the first pixel_atom information
    the_pixel->pix_atoms = malloc(sizeof*the_pixel->pix_atoms);
  }
  else
  {
    // otherwise reallocate memory to store the new pixel_atom information
    the_pixel->pix_atoms = realloc(the_pixel->pix_atoms, (the_pixel->patoms+1)*sizeof*the_pixel->pix_atoms);
  }
  the_pixel->pix_atoms[the_pixel->patoms].atom_id = atom_id;
  for ( axis = 0 ; axis < 3 ; axis ++ )
  {
    the_pixel->pix_atoms[the_pixel->patoms].coord[axis] = atom_coord[axis];
  }
  // increment the number of atom(s) in the pixel
  the_pixel->patoms ++;
}
```

A.4 C code: adjusting the number of pixels in the grid

```
float adjust_pixel_numbers (bool use_pbc, pixel_grid * grid, float cmin[3], float cmax[3], float pixel_size)
{

  int axis;                       // loop iterator axis id (1=x, 2=y, 3= z)
  float targetdp = 1.85;          // target atom density per pixel
  float rhonum;                   // number density
  float rhopix;                   // number density by pixel
  float mpsize;                   // modifier of the pixel size

  rhonum = atoms;
  if ( use_pbc )
  {
    // if PBC are used then use the exact volume
    rhonum = rhonum / volume;
  }
  else
  {
    // otherwise approximate a cubic box
    for ( axis = 0 ; axis < 3 ; axis ++ )      // For x, y and z
    {
      rhonum /= (cmax[axis] - cmin[axis]);
    }
  }

  // if the number density if < 0.01 atom / Å^3
  if (rhonum < 0.01)
  {
    rhopix = atoms;
    for ( axis = 0 ; axis < 3 ; axis ++ )      // For x, y and z
    {
      rhopix /= grid->n_pix[axis];
    }
    mpsize = pow (targetdp/rhopix, 1.0/3.0);
    if (mpsize > 1.0)
    {
      // we modify only the pixel size if the cutoff increases
      // otherwise we would increase the number of pixels
      pixel_size = pixel_size*mpsize;
      for ( axis = 0 ; axis < 3 ; axis ++ )
      {
        if ( use_pbc )
        {
          grid->n_pix[axis] = (int)(l_params[axis] / pixel_size);
        }
        else
        {
          grid->n_pix[axis] = (int)((cmax[axis] - cmin[axis]) / pixel_size);
        }
      }
   }
  }
  for ( axis = 0 ; axis < 3 ; axis ++ )         // For x, y and z
  {
    // correction if the number of pixel(s) on 'axis' is too small
    grid->n_pix[axis] = (grid->n_pix[axis] < 3) ? 1 : grid->n_pix[axis];
  }
  return pixel_size;
}
```

A.5 C code: preparation of the pixel grid

```
pixel_grid * prepare_pixel_grid (bool use_pbc)
{
  pixel_grid * grid;        // pointer to the pixel grid to create
  int axis;                 // loop iterator axis id (0=x, 1=y, 2=z)
  int aid;                  // loop iterator atom number (0, atoms--1)
  int pixel_num;            // pixel number in the grid
  int pixel_pos[3];         // pixel coordinates in the grid
  float cmin[3], cmax[3];   // float coordinates min, max values
  float f_coord[3];         // float fractional coordinates
  float pixel_size;         // the size of the pixel

  grid = malloc(sizeof*grid);
  pixel_size = cutoff + 0.5;                    // set a pixel size slightly higher than the cutoff
  if ( use_pbc )                                // using periodic boundary conditions
  {
    for ( axis = 0 ; axis < 3 ; axis ++ )       // For x, y and z
    {
      grid->n_pix[axis] = (int)(l_params[axis] / pixel_size); // number of pixel(s) on 'axis'
    }
  }
  else                                          // without periodic boundary conditions
  {
    for ( axis = 0 ; axis < 3 ; axis ++ ) cmin[axis] = cmax[axis] = c_coord[0][axis];
    for ( aid = 1 ; aid < atoms ; aid ++ )      // For all atoms
    {
      for ( axis = 0 ; axis < 3 ; axis ++ )     // For x, y and z
      {
        cmin[axis] = min(cmin[axis], c_coord[aid][axis]);
        cmax[axis] = max(cmax[axis], c_coord[aid][axis]);
      }
    }
    for ( axis = 0 ; axis < 3 ; axis ++ )       // For x, y and z
    {
      grid->n_pix[axis] = (int)((cmax[axis] - cmin[axis])/pixel_size); // number of pixel(s) on 'axis'
    }
  }
  pixel_size = adjust_pixel_numbers (use_pbc, grid, cmin, cmax, pixel_size);

  grid->n_xy = grid->n_pix[0] * grid->n_pix[1]; // number of pixels on the plan 'xy'
  grid->pixels = grid->n_xy * grid->n_pix[2];   // total number of pixels in the grid
  grid->pixel_list = malloc (grid->pixels*sizeof*grid->pixel_list);
  for ( pixel_num = 0 ; pixel_num < grid->pixels ; pixel_num ++ )
  {
    grid->pixel_list[pixel_num].pid = pixel_num;
    grid->pixel_list[pixel_num].patoms = 0;
    grid->pixel_list[pixel_num].tested = FALSE;
  }
  if ( use_pbc )                                // using periodic boundary conditions
  {
    for ( aid = 0 ; aid < atoms ; aid ++ )      // for all atoms
    {
      // with 'matrix_multiplication' a user defined function to perform the operation
      matrix_multiplication (f_coord, cart_to_frac, c_coord[aid]);
      for ( axis = 0 ; axis < 3 ; axis ++ )     // for x, y and z
      {
        f_coord[axis] = f_coord[axis] - floorf(f_coord[axis]);
        pixel_pos[axis] = (int)(f_coord[axis] * grid->n_pix[axis]);
      }
      pixel_num = pixel_pos[0] + pixel_pos[1] * grid->n_pix[0] + pixel_pos[2] * grid->n_xy;
      add_atom_to_pixel (& grid->pixel_list[pixel_num], pixel_pos, aid, f_coord);
    }
  }
  else                                          // without periodic boundary conditions
  {
    for ( aid = 0 ; aid < atoms ; aid ++ )      // for all atoms
    {
      for ( axis = 0 ; axis < 3 ; axis ++ )     // for x, y and z
      {
        pixel_pos[axis] = (grid->n_pix[axis] == 1) ? 0 : (int)((c_coord[aid][axis] - cmin[axis])/pixel_size);
      }
      pixel_num = pixel_pos[0] + pixel_pos[1] * grid->n_pix[0] + pixel_pos[2] * grid->n_xy;
      add_atom_to_pixel (& grid->pixel_list[pixel_num], pixel_pos, pixel_pos, aid, c_coord[aid]);
    }
  }
  return grid;
}
```

A.6 C code: finding pixel neighbors

```c
// finding neighbor pixels for pixel in the grid
// - bool use_pbc          : flag to set if PBC are used or not
// - pixel_grid * the_grid : pointer to the pixel grid
// - pixel * the_pix       : pointer to the pixel with neighbors to be found
void find_pixel_neighbors (bool use_pbc, pixel_grid * the_grid, pixel * the_pix)
{
  int axis;                       // loop iterator axis id (1=x , 2=y , 3= z)
  int x_pos, y_pos, z_pos;        // neighbor position on x, y and z
  int l_start[3] = { 0, 0, 0};    // loop iterators starting value
  int l_end[3] = { 3, 3, 3};      // loop iterators ending value
  int pmod[3] = {-1, 0, 1};       // position modifiers
  int nnp;                        // number of neighbors for pixel
  int nid;                        // neighbor id for pixel
  int pbc_shift[3][3][3];         // shift for pixel neighbor number due to PBC
  bool boundary = FALSE;          // is pixel on the boundary of the grid
  bool keep_neighbor = TRUE;      // keep or not neighbor during analysis

  if ( use_pbc )
  {
    // set PBC correction based on grid and pixel data
    set_pbc_shift (the_grid, the_pix->p_xyz, pbc_shift);
  }
  else
  {
    for ( axis = 0 ; axis < 3 ; axis ++ )
    {
      // if min or max on 'axis' then 'the_pix' is on the edge of the box
      if ( the_pix->p_xyz[axis] == 0 || the_pix->p_xyz[axis] == the_grid->n_pix[axis]-1 ) boundary = TRUE;
    }
  }
  for ( axis = 0 ; axis < 3 ; axis ++ )
  {
    if ( the_grid->n_pix[axis] == 1 )
    {
      // in the grid there is a single pixel in the 'axis' direction
      l_start[axis] = 1;
      l_end[axis] = 2;
    }
  }
  nnp = 0;
  for ( x_pos = l_start[0] ; x_pos < l_end[0] ; x_pos ++ )
  {
    for ( y_pos = l_start[1] ; y_pos < l_end[1] ; y_pos ++ )
    {
      for ( z_pos = l_start[2] ; z_pos < l_end[2] ; z_pos ++ )
      {
        keep_neighbor = TRUE;
        if ( ! use_pbc && boundary )
        {
          if (( the_pix->p_xyz[0] == 0 && x_pos == 0 ) || ( the_pix->p_xyz[0] == the_grid->n_pix[0]-1 && x_pos == 2 ))
          {
            keep_neighbor = FALSE;
          }
          else if (( the_pix->p_xyz[1] == 0 && y_pos == 0 ) || ( the_pix->p_xyz[1] == the_grid->n_pix[1]-1 && y_pos == 2 ))
          {
            keep_neighbor = FALSE;
          }
          else if (( the_pix->p_xyz[2] == 0 && z_pos == 0 ) || ( the_pix->p_xyz[2] == the_grid->n_pix[2]-1 && z_pos == 2 ))
          {
            keep_neighbor = FALSE;
          }
        }
        if ( keep_neighbor )
        {
          // evaluating neighbor pixel number in the grid
          nid = the_pix->pid + pmod[x_pos] + pmod[y_pos] * the_grid->n_pix[0] + pmod[z_pos] * the_grid->n_xy;
          // corrections if required in the case where PBC are applied
          if ( use_pbc ) nid += pbc_shift[x_pos][y_pos][z_pos];
          the_pix->pixel_neighbors[nnp] = nid;
          nnp ++ ;
        }
      }
    }
  }
  // total number of neighbors for pixel 'the_pix'
  the_pix->neighbors = nnp;
}
```

A.7 C code: inter-atomic distance calculation

```c
// evaluating the interatomic distance between 2 pixel atoms
// - bool use_pbc      : flag to set if PBC are used or not
// - pixel_atom * at_i : pointer to first pixel atom
// - pixel_atom * at_j : pointer to second pixel atom
distance evaluate_distance (bool use_pbc, pixel_atom * at_i, pixel_atom * at_j)
{
  int axis;          // axis loop iterator
  float u, v;        // float parameters
  distance dist;     // distance data to store calculation results
  for ( axis = 0 ; axis < 3 ; axis ++ )
  {
    dist.Rij[axis] = at_i->coord[axis] - at_j->coord[axis];
  }
  if ( use_pbc )
  {
    // then the pixel_atom's coordinates are in corrected fractional format
    for ( axis = 0 ; axis < 3 ; axis ++ )
    {
      dist.Rij[axis] = dist.Rij[axis] - roundf(dist.Rij[axis]);
    }
    // transform back to Cartesian coordinates
    // with 'matrix_multiplication' a user defined function to perform the operation
    matrix_multiplication (dist.Rij, frac_to_cart, dist.Rij);
  }
  dist.length = 0.0;
  for ( axis = 0 ; axis < 3 ; axis ++ )
  {
    dist.length += dist.Rij[axis] * dist.Rij[axis];
  }
  // returning the 'distance' data structure that contains:
  // - the squared value for Dij: no time consuming square root calculation !
  // - the components of the distance vector on x, y and z
  return dist;
}
```

A.8 C code: pixel search for first neighbor atoms

```
// searching for first neighbor atoms using the grid pixelation/partitioning method
// - bool use_pbc : flag to set if PBC are used or not
void pixel_search_for_neighbors (bool use_pbc)
{
  pixel_grid * all_pixels;    // pointer to the pixel grid for to analyze
  int pix, pjx;               // integer pixel ID numbers
  int aid, bid;               // integer loop atom numbers
  int pid;
  int l_start, l_end;         // integer loop modifier
  pixel * pix_i, * pix_j;     // pointers on pixel data structure
  pixel_atom * at_i, * at_j;  // pointers on pixel_atom data structure
  distance Dij;               // distance data structure

  all_pixels = prepare_pixel_grid (use_pbc);
  // note that the pixel grid 'all_pixels' must be prepared before the following
  // for all pixels in the grid
  for ( pix = 0 ; pix < all_pixels->pixels ; pix ++ )
  {
    // setting 'pix_i' as pointer to pixel number 'pix'
    pix_i = & all_pixels->pixel_list[pix];
    // if pixel 'pix_i' contains atom(s)
    if ( pix_i->patoms )
    {
      // search for neighbbor pixels of 'pix_i'
      find_pixel_neighbors ( use_pbc, all_pixels, pix_i );
      // testing all 'pix_i' neighbor pixels
      for ( pid = 0 ; pid < pix_i->neighbors ; pid ++ )
      {
        pjx = pix_i->pixel_neighbors[pid];
        // setting 'pix_j' as pointer to pixel number 'pjx'
        pix_j = & all_pixels->pixel_list[pjx];
        // checking pixel 'pix_j' if it:
        // - contains atom(s)
        // - was not tested, otherwise the analysis would have been performed already
        if ( pix_j->patoms && ! pix_j->tested )
        {
          // if 'pix_i' and 'pix_j' are the same, only test pair of different atoms
          l_end = (pjx != pix) ? 0 : 1;
          // for all atom(s) in 'pix'
          for ( aid = 0 ; aid < pix_i->patoms - l_end ; aid ++ )
          {
            // set pointer to the first atom to test
            at_i = & pix_i->pix_atoms[aid];
            l_start = (pjx != pix) ? 0 : aid + 1;
            // for all atom(s) in 'pix_j'
            for ( bid = l_start ; bid < pix_j->patoms ; bid ++ )
            {
              // set pointer to the second atom to test
              at_j = & pix_j->pix_atoms[bid];
              // evaluate interatomic distance
              Dij = evaluate_distance (use_pbc, at_i, at_j);
              if ( Dij.length < cutoff_squared )
              {
                // this is a first neighbor bond !
              }
            }
          }
        }
      }
      // store that pixel 'pix' was tested
      pix_i->tested = TRUE;
    }
  }
}
```

Appendix B Commented FORTRAN90 code

B.1 FORTRAN90 code: data structures and global variables

```
! global data structures and variables used in the next code sections
MODULE parameters

  ! atom in pixel data structure
  TYPE pixel_atom
    INTEGER                                       :: atom_id            ! the atom ID
    REAL, DIMENSION(3)                            :: coord              ! the atom coordinates on x, y and z
  END TYPE pixel_atom

  ! pixel data structure
  TYPE pixel
    INTEGER                                       :: pid                ! the pixel number
    INTEGER, DIMENSION(3)                         :: p_xyz              ! the pixel coordinates in the grid
    LOGICAL                                       :: tested             ! was the pixel checked already
    INTEGER                                       :: patoms             ! number of atom(s) in pixel
    TYPE(pixel_atom), DIMENSION(:), ALLOCATABLE   :: pix_atoms          ! list of atom(s) in pixel, to be allocated
    INTEGER                                       :: neighbors          ! number of neighbors for pixel
    INTEGER, DIMENSION(27)                        :: pixel_neighbors    ! the list of neighbor pixels, maximum 27
  END TYPE pixel

  ! pixel grid data structure
  TYPE pixel_grid
    INTEGER                                       :: pixels             ! total number of pixels in the grid
    INTEGER, DIMENSION(3)                         :: n_pix              ! number of pixel(s) on each axis
    INTEGER                                       :: n_xy               ! number of pixels in the plan xy
    TYPE (pixel), DIMENSION(:), ALLOCATABLE, TARGET :: pixel_list       ! pointer to the pixels, to be allocated
  END TYPE pixel_grid

  ! distance data structure
  TYPE distance
    REAL                                          :: length             ! the distance in \AA\ squared
    REAL, DIMENSION(3)                            :: Rij                ! vector components of x, y and z
  END TYPE distance

  !
  ! the following are considered to be provided by the user
  !

  ! model description
  INTEGER                                         :: atoms              ! the total number of atom(s)
  REAL, DIMENSION(atoms,3)                        :: c_coord            ! list of Cartesian coordinates
  REAL                                            :: cutoff             ! the cutoff to define first neighbors
  REAL                                            :: cutoff_squared     ! squared value of the cutoff to define first neighbors

  ! model box description, if PBC are applied
  REAL                                            :: volume             ! the lattice volume
  REAL, DIMENSION(3)                              :: l_params           ! lattice a, b and c
  REAL, DIMENSION(3,3)                            :: cart_to_frac       ! Cartesian to fractional coordinates matrix
  REAL, DIMENSION(3,3)                            :: frac_to_cart       ! fractional to Cartesian coordinates matrix

END MODULE parameters
```

B.2 FORTRAN90 code: set pixel periodic boundary condition shift

```
SUBROUTINE set_pbc_shift (grid, pixel_coord, pbc_shift)

  USE parameters
  IMPLICIT NONE

  TYPE (pixel_grid), INTENT(IN)            :: grid                ! the pixel grid
  INTEGER, DIMENSION(3), INTENT(IN)        :: pixel_coord         ! the pixel coordinates in the grid
  INTEGER, DIMENSION(3,3,3), INTENT(INOUT) :: pbc_shift           ! the shift, correction, to be calculated

  pbc_shift(:,:,:) = 0                                            ! initialization without any shift

  if ( pixel_coord(1) .eq. 0 ) then                               ! pixel position on 'x' is min
    pbc_shift(1,:,:) = grid%n_pix(1)
  else if ( pixel_coord(1) .eq. grid%n_pix(1)-1 ) then            ! pixel position on 'x' is max
    pbc_shift(3,:,:) = - grid%n_pix(1)
  endif

  if ( pixel_coord(2) .eq. 0 ) then                               ! pixel position on 'y' is min
    pbc_shift(:,1,:) = pbc_shift(:,1,:) + grid%n_xy
  else if ( pixel_coord(2) .eq. grid%n_pix(2)-1 ) then            ! pixel position on 'y' is max
    pbc_shift(:,3,:) = pbc_shift(:,3,:) - grid%n_xy
  endif

  if ( pixel_coord(3) .eq. 0 ) then                               ! pixel position on 'z' is min
    pbc_shift(:,:,1) = pbc_shift(:,:,1) + grid%pixels
  else if ( pixel_coord(3) .eq. grid%n_pix(3)-1 ) then            ! pixel position on 'z' is max
    pbc_shift(:,:,3) = pbc_shift(:,:,3) - grid%pixels
  endif

END SUBROUTINE set_pbc_shift
```

B.3 FORTRAN90 code: add atom to pixel

```
SUBROUTINE add_atom_to_pixel (the_pixel, pixel_coord, atom_id, atom_coord)

  USE parameters
  IMPLICIT NONE

  TYPE (pixel), INTENT(INOUT)                 :: the_pixel   ! pointer to the pixel
  INTEGER, DIMENSION(3), INTENT(IN)           :: pixel_coord ! the pixel coordinates in the grid
  INTEGER, INTENT(IN)                         :: atom_id     ! the atom ID number
  INTEGER                                     :: axis        ! loop iterator axis id (1=x, 2=y, 3=z)
  REAL, DIMENSION(3), INTENT(IN)              :: atom_coord  ! the atomic coordinates
  TYPE(pixel_atom), DIMENSION(:), ALLOCATABLE :: tmp_atoms

  if (the_pixel%patoms .eq. 0) then
    ! if the pixel do not contains any atom yet, then save its coordinates in the grid
    do axis = 1 , 3
      the_pixel%p_xyz(axis) = pixel_coord(axis)
    enddo
     ! allocate the memory to store the first pixel_atom information
    allocate(the_pixel%pix_atoms(1))
  else
    ! otherwise reallocate memory to store the new pixel_atom information
    allocate(tmp_atoms(the_pixel%patoms+1))
    tmp_atoms(1:the_pixel%patoms) = the_pixel%pix_atoms
    call move_alloc (tmp_atoms, the_pixel%pix_atoms)
    deallocate (tmp_atoms)
  endif
  ! increment the number of atom(s) in the pixel
  the_pixel%patoms = the_pixel%patoms + 1
  the_pixel%pix_atoms(the_pixel%patoms)%atom_id = atom_id
  do axis = 1 , 3
    the_pixel%pix_atoms(the_pixel%patoms)%coord(axis) = atom_coord(axis)
  enddo

END SUBROUTINE add_atom_to_pixel
```

B.4 FORTRAN90 code: adjusting the number of pixels in the grid

```
SUBROUTINE adjust_pixel_numbers (use_pbc, grid, cmin, cmax, pixel_size)

  USE parameters
  IMPLICIT NONE

  LOGICAL, INTENT(IN)              :: use_pbc          ! flag to set if PBC are used or not
  TYPE (pixel_grid), INTENT(INOUT) :: grid             ! the pixel grid to prepare
  REAL, DIMENSION(3), INTENT(IN)   :: cmin, cmax       ! real precision coordinates min, max
  REAL, INTENT(INOUT)              :: pixel_size       ! the initial pixel size
  REAL                             :: targetdp=1.85    ! target atom density per pixel
  REAL                             :: rhonum           ! number density
  REAL                             :: rhopix           ! number density by pixel
  REAL                             :: mpsize           ! modifier of the pixel size
  INTEGER                          :: axis             ! loop iterator axis id (1=x, 2=y, 3=z)

  rhonum = atoms
  if (use_pbc) then
    ! if PBC are used then use the exact volume
    rhonum = rhonum / volume
  else
    ! otherwise approximate a cubic box
    do axis = 1 , 3                    ! for x, y and z
      rhonum = rhonum / (cmax(axis) - cmin(axis))
    enddo
  endif

  ! if the number density if < 0.01 atom / Å^3
  if (rhonum .lt. 0.01) then

    rhopix = atoms
    do axis = 1 , 3                    ! for x, y and z
      rhopix = rhopix / grid%n_pix(axis)
    enddo
    mpsize = (targetdp/rhopix)**(1.0/3.0)
    if (mpsize .gt. 1.0) then
      ! we modify only the pixel size if the cutoff increases
      ! otherwise we would increase the number of pixels
      pixel_size = pixel_size*mpsize
      do axis= 1 , 3                   ! for x, y and z
        if ( use_pbc ) then
          grid%n_pix(axis) = INT(l_params(axis) / pixel_size)
        else
          grid%n_pix(axis) = INT((cmax(axis) - cmin(axis)) / pixel_size)
        endif
      endif
    endif

  endif

  do axis = 1 , 3                      ! for x, y and z
    ! correction if the number of pixel(s) on 'axis' is too small
    if ( grid%n_pix(axis) .lt. 3 ) then
      grid%n_pix(axis) = 1
    endif
  enddo

END SUBROUTINE adjust_pixel_numbers
```

B.5 FORTRAN90 code: preparation of the pixel grid

```
! preparation of the pixel grid
SUBROUTINE prepare_pixel_grid (use_pbc, grid)

  USE parameters
  IMPLICIT NONE

  LOGICAL, INTENT(IN)              :: use_pbc          ! flag to set if PBC are used or not
  TYPE (pixel_grid), INTENT(INOUT) :: grid             ! the pixel grid to prepare
  INTEGER                          :: axis             ! loop iterator axis id (1=x, 2=y, 3=z)
  INTEGER                          :: aid              ! loop iterator atom number (1, atoms)
  INTEGER                          :: pixel_num        ! pixel number in the grid
  INTEGER, DIMENSION(3)            :: pixel_pos        ! pixel coordinates in the grid
  REAL, DIMENSION(3)               :: cmin, cmax       ! real precision coordinates min, max
  REAL, DIMENSION(3)               :: f_coord          ! real precision fractional coordinates
  REAL                             :: pixel_size       ! the size of the pixel

  pixel_size = cutoff + 0.5                            ! set a pixel size slightly higher than the cutoff
  if ( use_pbc ) then                                  ! using periodic boundary conditions
      do axis = 1 , 3                                  ! for x, y and z
      ! number of pixel(s) on 'axis'
      grid%n_pix(axis) = INT(l_params(axis) / pixel_size)
    enddo
  else                                                 ! without periodic boundary conditions
    do axis = 1 , 3
      cmin(axis) = c_coord(1,axis)
      cmax(axis) = c_coord(1,axis)
    enddo
    do aid = 2 , atoms                                 ! for all atoms
      do axis = 1 , 3                                  ! for x, y and z
        cmin(axis) = min(cmin(axis), c_coord(aid,axis))
        cmax(axis) = max(cmax(axis), c_coord(aid,axis))
      enddo
    enddo
    do axis = 1 , 3                                    ! for x, y and z
      ! number of pixel(s) on 'axis'
      grid%n_pix(axis) = INT((cmax(axis) - cmin(axis))/pixel_size)
    enddo
  endif

  call adjust_pixel_numbers (use_pbc, grid, cmin, cmax, pixel_size)

  grid%n_xy = grid%n_pix(1) * grid%n_pix(2)            ! number of pixels on the plan 'xy'
  grid%pixels = grid%n_xy * grid%n_pix(3)              ! total number of pixels in the grid

  allocate (grid%pixel_list(grid%pixels))
  do pixel_num = 1 , grid%pixels
    grid%pixel_list(pixel_num)%pid = pixel_num
    grid%pixel_list(pixel_num)%patoms = 0
    grid%pixel_list(pixel_num)%tested = .false.
  enddo
  if ( use_pbc ) then                                  ! using periodic boundary conditions
  do aid = 1 , atoms                                   ! for all atoms
     f_coord = MATMUL ( c_coord(aid,:), cart_to_frac )
     do axis = 1 , 3                                   ! for x, y and z
       f_coord(axis) = f_coord(axis) - floor(f_coord(axis))
       pixel_pos(axis) = INT(f_coord(axis) * grid%n_pix(axis))
     enddo
     pixel_num = pixel_pos(1) + pixel_pos(2) * grid%n_pix(1) + pixel_pos(3) * grid%n_xy
     call add_atom_to_pixel (grid%pixel_list(pixel_num+1), pixel_pos, aid, f_coord)
    enddo
  else                                                 ! without periodic boundary conditions
    do aid = 1 , atoms                                 ! for all atoms
      do axis = 1 , 3                                  ! for x, y and z
        if (grid%n_pix(axis) .eq. 1) then
          pixel_pos(axis) = 0
        else
          pixel_pos(axis) = INT((c_coord(aid,axis) - cmin(axis))/pixel_size)
        endif
      enddo
      pixel_num = pixel_pos(1) + pixel_pos(2) * grid%n_pix(1) + pixel_pos(3) * grid%n_xy
      call add_atom_to_pixel (grid%pixel_list(pixel_num+1), pixel_pos, aid, c_coord(aid,:))
    enddo
  endif

END SUBROUTINE prepare_pixel_grid
```

B.6 FORTRAN90 code: finding pixel neighbors

```
SUBROUTINE find_pixel_neighbors (use_pbc, the_grid, the_pix)

  USE parameters
  IMPLICIT NONE

  LOGICAL, INTENT(IN)                  :: use_pbc                ! flag to set if PBC are used or not
  TYPE (pixel_grid), INTENT(INOUT)     :: the_grid               ! pointer to the pixel grid
  TYPE (pixel), POINTER, INTENT(INOUT) :: the_pix                ! pointer to the pixel
  INTEGER                              :: axis                   ! loop iterator axis id (1=x, 2=y, 3=z)
  INTEGER                              :: x_pos, y_pos, z_pos    ! neighbor position on x, y and z
  INTEGER, DIMENSION(3)                :: l_start = (/1, 1, 1/)  ! loop iterators starting value
  INTEGER, DIMENSION(3)                :: l_end = (/3, 3, 3/)    ! loop iterators ending value
  INTEGER, DIMENSION(3)                :: pmod = (/-1, 0, 1/)    ! position modifiers
  INTEGER                              :: nnp                    ! number of neighbors for pixel
  INTEGER                              :: nid                    ! neighbor id for pixel
  INTEGER, DIMENSION(3,3,3)            :: pbc_shift              ! shift, correction, due to PBC
  LOGICAL                              :: boundary=.false.       ! is pixel on the boundary of the grid
  LOGICAL                              :: keep_neighbor = .true. ! keep or not neighbor during analysis

  if ( use_pbc ) then
    call set_pbc_shift (the_grid, the_pix%p_xyz, pbc_shift)
  else
    do axis = 1 , 3
      if ( the_pix%p_xyz(axis) .eq. 0 .or. the_pix%p_xyz(axis) .eq. the_grid%n_pix(axis) - 1 ) then
        boundary = .true.
      endif
    enddo
  endif
  do axis = 1 , 3
    if ( the_grid%n_pix(axis) .eq. 1 ) then
      ! in the grid there is a single pixel in the 'axis' direction
      l_start(axis) = 2
      l_end(axis) = 2
    endif
  enddo
  nnp = 0
  do x_pos = l_start(1) , l_end(1)
    do y_pos = l_start(2) , l_end(2)
      do z_pos = l_start(3) , l_end(3)
        keep_neighbor = .true.
        if ( .not. use_pbc .and. boundary ) then
          if ( the_pix%p_xyz(1) .eq. 0 .and. x_pos .eq. 1 ) then
            keep_neighbor = .false.
          else if ( the_pix%p_xyz(1) .eq. the_grid%n_pix(1)-1 .and. x_pos .eq. 3 ) then
            keep_neighbor = .false.
          else if ( the_pix%p_xyz(2) .eq. 0 .and. y_pos .eq. 1 ) then
            keep_neighbor = .false.
          else if ( the_pix%p_xyz(2) .eq. the_grid%n_pix(2)-1 .and. y_pos .eq. 3 ) then
            keep_neighbor = .false.
          else if ( the_pix%p_xyz(3) .eq. 0 .and. z_pos .eq. 1 ) then
            keep_neighbor = .false.
          else if ( the_pix%p_xyz(3) .eq. the_grid%n_pix(3)-1 .and. z_pos .eq. 3 ) then
            keep_neighbor = .false.
          endif
        endif
        if ( keep_neighbor ) then
          ! evaluating neighbor pixel number in the grid
          nid = the_pix%pid + pmod(x_pos) + pmod(y_pos) * the_grid%n_pix(1) + pmod(z_pos) * the_grid%n_xy
          if ( use_pbc ) then
            ! corrections if required in the case where PBC are applied
            nid = nid + pbc_shift(x_pos, y_pos, z_pos)
          endif
          the_pix%pixel_neighbors(nnp) = nid
          nnp = nnp + 1
        endif
      enddo
    enddo
  enddo
  ! total number of neighbors for pixel 'the_pix'
  the_pix%neighbors = nnp

END SUBROUTINE find_pixel_neighbors
```

B.7 FORTRAN90 code: inter-atomic distance calculation

```
! evaluating the interatomic distance between 2 pixel atoms
SUBROUTINE evaluate_distance (use_pbc, at_i, at_j, dist)

  USE parameters
  IMPLICIT NONE

  LOGICAL, INTENT(IN)                    :: use_pbc   ! flag to set if PBC are used or not
  TYPE (pixel_atom), POINTER, INTENT(IN) :: at_i      ! first pixel atom
  TYPE (pixel_atom), POINTER, INTENT(IN) :: at_j      ! second pixel atom
  TYPE (distance), INTENT(INOUT)         :: dist      ! calculation results
  INTEGER                                :: axis      ! loop iterator axis id (1=x, 2=y, 3=z)

  do axis = 1 , 3
    dist%Rij(axis) = at_i%coord(axis) - at_j%coord(axis)
  enddo
  if ( use_pbc ) then
    ! then the pixel_atom's coordinates are in corrected fractional format
    do axis = 1 , 3
      dist%Rij(axis) = dist%Rij(axis) - AnINT(dist%Rij(axis))
    enddo
    ! transform back to Cartesian coordinates
    dist%Rij = MATMUL( dist%Rij, frac_to_cart )
  endif

  dist%length = 0.0
  do axis = 1 , 3
    dist%length = dist%length + dist%Rij(axis) * dist%Rij(axis)
  enddo
  ! returning the 'distance' data structure that contains:
  ! - the squared value for Dij: no time consuming square root calculation !
  ! - the components of the distance vector on x, y and z
END SUBROUTINE evaluate_distance
```

B.8 FORTRAN90 code: pixel search for first neighbor atoms

```
SUBROUTINE pixel_search_for_neighbors (use_pbc)

  USE parameters
  IMPLICIT NONE

  LOGICAL, INTENT(IN)   :: use_pbc                    ! flag to set if PBC are used or not
  TYPE (pixel_grid)     :: all_pixels                 ! the pixel grid to analyze
  INTEGER               :: pix, pjx                   ! integer pixel ID numbers
  INTEGER               :: aid, bid                   ! integer loop atom numbers
  INTEGER               :: l_start, l_end             ! integer loop modifier
  INTEGER               :: pid
  TYPE (pixel_atom), POINTER :: pix_i, pix_j          ! pointers on pixel data structure
  TYPE (atom), POINTER  :: at_i, at_j                 ! pointers on pixel_atom data structure
  TYPE (distance)       :: Dij                        ! distance data structure

  call prepare_pixel_grid (use_pbc, all_pixels)
  ! note that the pixel grid 'all_pixels' must be prepared before the following
  ! for all pixels in the grid
  do pix = 1 , all_pixels%pixels
    ! setting 'pix_i' as pointer to pixel number 'pix'
    pix_i => all_pixels%pixel_list(pix)
    ! if pixel 'pix_i' contains atom(s)
    if ( pix_i%patoms .gt. 0 ) then

      ! search for pixel neighbbors of 'pix_'
      call find_pixel_neighbors (use_pbc, all_pixels, pix_i)

      ! testing all 'pix_i' neighbor pixels
      do pid = 1 , pix_i%neighbors
        pjx = pix_i%pixel_neighbors(pid)
        ! setting 'pix_j' as pointer to pixel number 'pjx'
        pix_j => all_pixels%pixel_list(pjx)
        ! checking pixel 'pix_j' if it:
        ! - contains atom(s)
        ! - was not tested, otherwise the analysis would have been performed already
        if ( pix_j%patoms .gt. 0 .and. .not. pix_j%tested ) then
          ! If 'pix_i' and 'pix_j' are the same, only test pair of different atoms
          if ( pjx .eq. pix ) then
            l_end = 1
          else
            l_end = 0
          endif
          ! for all atom(s) in 'pix_i'
          do aid = 1 , pix_i%patoms - l_end
            ! set pointers to the first atom to test
            at_i => pix_i%pix_atoms(aid)
            if ( pjx .eq. pix ) then
              l_start = aid + 1
            else
              l_start = 1
            endif
            ! for all atom(s) in 'pix_j'
            do bid = l_start , pix_j%patoms
              ! set pointers to the second atom to test
              at_j => pix_j%pix_atoms(bid)
              ! evaluate interatomic distance
              call evaluate_distance (use_pbc, at_i, at_j, Dij)
              if ( Dij%length .lt. cutoff_squared ) then
                ! this is a first neighbor bond !
              endif
            enddo
          enddo
        endif
      enddo
      ! store that pixel 'pix_i' was tested
      pix_i%tested = .true.
    endif
  enddo

END SUBROUTINE pixel_search_for_neighbors
```

Appendix C Commented Python code

C.1 Python code: data structures and global variables

```python
# Global data structures and variables used in the next code sections
import numpy as np

# atom in pixel data structure
class PixelAtom:
  def __init__(self, atom_id=0, coord=None):
    self.atom_id = atom_id                                          # the atom ID
    self.coord = np.zeros(3) if coord is None else np.array(coord)  # the atom coordinates on x, y and z

# pixel data structure
class Pixel:
  def __init__(self, pid=0, p_xyz=None, tested=False, patoms=0, pix_atoms=None, neighbors=0):
    self.pid = pid                                                  # the pixel number
    self.p_xyz = np.zeros(3) if p_xyz is None else np.array(p_xyz)  # the pixel coordinates in the grid
    self.tested = tested                                            # was the pixel checked already
    self.patoms = patoms                                            # number of atom(s) in pixel
    self.pix_atoms = [] if pix_atoms is None else pix_atoms         # list of atom(s) in the pixel
    self.neighbors = neighbors                                      # number of neighbors for pixel
    self.pixel_neighbors = np.zeros(27, dtype=int)                  # the list of neighbor pixels, maximum 27

# pixel grid data structure
class PixelGrid:
  def __init__(self, pixels=0, n_pix=None, n_xy=0, pixel_list=None):
    self.pixels = pixels                                            # total number of pixels in the grid
    self.n_pix = np.zeros(3, dtype=int) if n_pix is None else np.array(n_pix) # pixel(s) on each axis
    self.n_xy = n_xy                                                # number of pixels in the plan xy
    self.pixel_list = [] if pixel_list is None else pixel_list      # pointer to the pixels, to be allocated

# bond distance data structure
class Distance:
  def __init__(self, length=0.0, Rij=None):
    self.length = length                                            # the distance in Å squared
    self.Rij = np.zeros(3) if Rij is None else np.array(Rij)        # vector components of x, y and z

#
# the following are considered to be provided by the user
#

# model description
atoms = 0                                                           # the total number of atom(s)
c_coord = None                                                      # list of Cartesian coordinates: c_coord[atoms][3]
cutoff = 0.0                                                        # the cutoff to define atomic bond(s)
cutoff_squared = 0.0                                                # squared value for the cutoff

# model box description, if PBC are applied
volume = 0.0                                                        # the lattice volume
l_params = np.zeros(3)                                              # lattice a, b and c
cart_to_frac = np.zeros((3, 3))                                     # Cartesian to fractional coordinates matrix
frac_to_cart = np.zeros((3, 3))                                     # fractional to Cartesian coordinates matrix
```

C.2 Python code: set pixel periodic boundary condition shift

```
# adjust, if needed, the shift to search for pixel neighbor(s) using PBC
# - PixelGrid pixel_grid     : the pixel grid
# - int pixel_coord[3]       : the pixel coordinates in the grid
# - int pbc_shift[3][3][3]   : the shift, correction, to be calculated
def set_pbc_shift(pixel_grid : PixelGrid, pixel_coord : np.ndarray, pbc_shift : np.ndarray):
  # Initialize pbc_shift to zero
  for x_pos in range(3):
    for y_pos in range(3):
      for z_pos in range(3):
        pbc_shift[x_pos][y_pos][z_pos] = 0        # initialization without any shift

  if pixel_coord[0] == 0:                         # pixel position on 'x' is min
    for y_pos in range(3):
      for z_pos in range(3):
        pbc_shift[0][y_pos][z_pos] = pixel_grid.n_pix[0]

  elif pixel_coord[0] == pixel_grid.n_pix[0] - 1: # pixel position on 'x' is max
    for y_pos in range(3):
      for z_pos in range(3):
        pbc_shift[2][y_pos][z_pos] = -pixel_grid.n_pix[0]

  if pixel_coord[1] == 0:                         # pixel position on 'y' is min
    for x_pos in range(3):
      for z_pos in range(3):
        pbc_shift[x_pos][0][z_pos] += pixel_grid.n_xy

  elif pixel_coord[1] == pixel_grid.n_pix[1] - 1: # pixel position on 'y' is max
    for x_pos in range(3):
      for z_pos in range(3):
        pbc_shift[x_pos][2][z_pos] -= pixel_grid.n_xy

  if pixel_coord[2] == 0:                         # pixel position on 'z' is min
    for x_pos in range(3):
      for y_pos in range(3):
        pbc_shift[x_pos][y_pos][0] += pixel_grid.pixels

  elif pixel_coord[2] == pixel_grid.n_pix[2] - 1: # pixel position on 'z' is max
    for x_pos in range(3):
      for y_pos in range(3):
        pbc_shift[x_pos][y_pos][2] -= pixel_grid.pixels
```

C.3 Python code: add atom to pixel

```
def add_atom_to_pixel(the_pixel: Pixel, pixel_coord: np.ndarray, atom_id: int, atom_coord: np.ndarray):
  if not the_pixel.patoms:
    # if the pixel do not contains any atom yet, then save its coordinates in the grid
    for axis in range(3):
      the_pixel.p_xyz[axis] = pixel_coord[axis]

  new_atom = PixelAtom(atom_id, atom_coord)
  the_pixel.pix_atoms.append(new_atom)

  # increment the number of atom(s) in the pixel
  the_pixel.patoms += 1
```

C.4 Python code: adjusting the number of pixels in the grid

```python
def adjust_pixel_numbers (use_pbc : bool, grid : PixelGrid, cmin : np.ndarray, cmax : np.ndarray, pixel_size : float):

  targetdp = 1.85  # target atom density per pixel
  rhonum = atoms
  if use_pbc:
    rhonum = rhonum / volume
  else:
    for axis in range(3):                          # for x, y and z
      rhonum = rhonum / (cmax[axis] - cmin[axis])

  if rhonum < 0.01:
    rhopix = atoms
    for axis in range(3):                          # for x, y and z
      rhopix = rhopix / grid.n_pix[axis]

    mpsize = (targetdp / rhopix ) **(1.0/3.0)
    if mpsize > 1.0:
      # we modify only the pixel size if the cutoff increases
      # otherwise we would increase the number of pixels
      pixel_size = pixel_size * mpsize
      for axis in range(3):                        # for x, y and z
        if use_pbc:
          grid.n_pix[axis] = int(l_params[axis] / pixel_size)
        else
          grid.n_pix[axis] = int((cmax[axis] - cmin[axis]) / pixel_size)

  for axis in range(3):                            # for x, y and z
    # correction if the number of pixels on 'axis' is too small
    grid.n_pix[axis] = 1 if grid.n_pix[axis] < 3 else grid.n_pix[axis]

  return pixel_size
```

C.5 Python code: preparation of the pixel grid

```python
# preparation of the pixel grid
# - bool use_pbc : flag to set if PBC are used or not
def prepare_pixel_grid(use_pbc : bool):
  grid = PixelGrid()                                # create a new pixel grid
  cmin = [float('inf')] * 3                         # initialize to infinity
  cmax = [-float('inf')] * 3                        # initialize to negative infinity
  pixel_pos = np.zeros(3, dtype=int)
  pixel_size = cutoff + 0.5;                        # set a pixel size slightly higher than the cutoff

  if use_pbc:                                       # using periodic boundary conditions
    for axis in range(3):                           # for x, y and z
      # number of pixels on 'axis'
      grid.n_pix[axis] = int(l_params[axis] / pixel_size)
  else:                                             # without periodic boundary conditions
    for axis in range(3):
      cmin[axis] = cmax[axis] = c_coord[0][axis]
    for aid in range(1, atoms):                     # for all atoms
      for axis in range(3):                         # for x, y and z
        cmin[axis] = min(cmin[axis], c_coord[aid][axis])
        cmax[axis] = max(cmax[axis], c_coord[aid][axis])
    for axis in range(3):                           # for x, y and z
      # number of pixels on 'axis'
      grid.n_pix[axis] = int((cmax[axis] - cmin[axis]) / pixel_size)

  pixel_size = adjust_pixel_numbers (use_pbc, grid, cmin, cmax, pixel_size)

  grid.n_xy = grid.n_pix[0] * grid.n_pix[1]         # number of pixels on the plan 'xy'
  grid.pixels = grid.n_xy * grid.n_pix[2]           # total number of pixels in the grid

  grid.pixel_list = [Pixel(pid=i) for i in range(grid.pixels)]

  if use_pbc:                                       # using periodic boundary conditions
    for aid in range(atoms):                        # for all atoms
      # with 'matrix_multiplication' a user defined function to perform the operation
      f_coord = matrix_multiplication(cart_to_frac, c_coord[aid])
      for axis in range(3):                         # for x, y and z
        f_coord[axis] = f_coord[axis] - np.floor(f_coord[axis])
        pixel_pos[axis] = int(f_coord[axis] * grid.n_pix[axis])
      pixel_num = pixel_pos[0] + pixel_pos[1] * grid.n_pix[0] + pixel_pos[2] * grid.n_xy
      add_atom_to_pixel(grid.pixel_list[pixel_num], pixel_pos, aid, f_coord)
  else:                                             # without periodic boundary conditions
    for aid in range(atoms):                        # for all atoms
      for axis in range(3):                         # for x, y and z
        pixel_pos[axis] = int((c_coord[aid][axis] - cmin[axis]) / pixel_size)
      pixel_num = pixel_pos[0] + pixel_pos[1] * grid.n_pix[0] + pixel_pos[2] * grid.n_xy
      add_atom_to_pixel(grid.pixel_list[pixel_num], pixel_pos, aid, c_coord[aid])

  return grid
```

C.6 Python code: finding pixel neighbors

```python
# Finding neighbor pixels for pixel in the grid
# - bool use_pbc         : flag to set if PBC are used or not
# - PixelGrid * the_grid : pointer to the pixel grid
# - Pixel * the_pix      : pointer to the pixel with neighbors to be found
def find_pixel_neighbors(use_pbc : bool, the_grid : PixelGrid, the_pix : Pixel):
  boundary = False                          # is pixel on the boundary of the grid
  keep_neighbor = True                      # keep or not neighbor during analysis
  l_start = [0, 0, 0]                       # loop iterators starting value
  l_end = [3, 3, 3]                         # loop iterators ending value
  pmod = [-1, 0, 1]                         # position modifiers
  pbc_shift = np.zeros((3, 3, 3), dtype=int)  # shift for pixel neighbor number due to PBC

  # check if PBC are used
  if use_pbc:
    # set PBC correction based on grid and pixel data
    set_pbc_shift(the_grid, the_pix.p_xyz, pbc_shift)
  else:
    for axis in range(3):
      if the_pix.p_xyz[axis] == 0 or the_pix.p_xyz[axis] == the_grid.n_pix[axis] - 1:
        # if min or max on 'axis' then 'the_pix' is on the edge of the box
        boundary = True

  # adjust the loop start and end based on the grid dimensions
  for axis in range(3):
    if the_grid.n_pix[axis] == 1:
      # in the grid there is a single pixel in the 'axis' direction
      l_start[axis] = 1
      l_end[axis] = 1

  nnp = 0  # number of neighbors
  for x_pos in range(l_start[0], l_end[0]):
    for y_pos in range(l_start[1], l_end[1]):
      for z_pos in range(l_start[2], l_end[2]):
        keep_neighbor = True

        if not use_pbc and boundary:
          if the_pix.p_xyz[0] == 0 and x_pos == 0:
            keep_neighbor = False
          elif the_pix.p_xyz[0] == the_grid.n_pix[0]-1 and x_pos == 2:
            keep_neighbor = False
          elif the_pix.p_xyz[1] == 0 and y_pos == 0:
            keep_neighbor = False
          elif the_pix.p_xyz[1] == the_grid.n_pix[1]-1 and y_pos == 2:
            keep_neighbor = False
          elif the_pix.p_xyz[2] == 0 and z_pos == 0:
            keep_neighbor = False
          elif the_pix.p_xyz[2] == the_grid.n_pix[2]-1 and z_pos == 2:
            keep_neighbor = False

        if keep_neighbor:
          # evaluating neighbor pixel number in the grid
          nid = the_pix.pid + pmod[x_pos] + pmod[y_pos] * the_grid.n_pix[0] + pmod[z_pos] * the_grid.n_xy
          if use_pbc:
      # corrections if required in the case where PBC are applied
      nid += pbc_shift[x_pos][y_pos][z_pos]
          the_pix.pixel_neighbors[nnp] = nid
          nnp += 1

  # total number of neighbors for pixel 'the_pix'
  the_pix.neighbors = nnp
```

C.7 Python code: inter-atomic distance calculation

```python
# evaluating the interatomic distance between 2 pixel atoms
def evaluate_distance(use_pbc : bool, at_i : PixelAtom, at_j : PixelAtom):
  dist = Distance()   # placeholder for the distance data structure
  Rij = np.zeros(3)  # initialize the distance vector
  # calculating the distance components between atoms
  for axis in range(3):
    Rij[axis] = at_i.coord[axis] - at_j.coord[axis]

  if use_pbc:
    # then the PixelAtom's coordinates are in corrected fractional format
    for axis in range(3):
      Rij[axis] = Rij[axis] - round(Rij[axis])

    # transform back to Cartesian coordinates
    # matrix_multiplication is assumed to be defined elsewhere
    Rij = matrix_multiplication(frac_to_cart, Rij)

  # calculating the squared distance (no square root for efficiency)
  dist.length = np.sum(Rij ** 2)
  dist.Rij = Rij  # store the distance vector components

  # returning the 'distance' data structure that contains:
  # - the squared value for Dij
  # - the components of the distance vector on x, y, and z
  return dist
```

C.8 Python code: pixel search for first neighbor atoms

```python
def pixel_search_for_neighbors(use_pbc : bool):
  # pointer to the pixel grid for analysis
  all_pixels = prepare_pixel_grid(use_pbc)

  # for all pixels in the grid
  for pix in range(all_pixels.pixels):
    # setting pix_i as pointer to pixel number pix
    pix_i = all_pixels.pixel_list[pix]

    # if pixel pix_i contains atom(s)
    if pix_i.patoms:
      # search for neighbor pixels
      find_pixel_neighbors(use_pbc, all_pixels, pix_i)

      # testing all pix_i neighbor pixels
      for pid in range(pix_i.neighbors):
        pjx = pix_i.pixel_neighbors[pid]
        # Setting pix_j as pointer to pixel number pjx
        pix_j = all_pixels.pixel_list[pjx]

        # checking pixel pix_j if it:
        # - contains atom(s)
        # - was not tested, otherwise the analysis would have been performed already
        if pix_j.patoms and not pix_j.tested:
          # If pix_i and pix_j are the same, only test pair of different atoms
          end = 0 if pjx != pix else 1

          # for all atom(s) in pix_i
          for aid in range(pix_i.patoms - end):
            # Set pointer to the first atom to test
            at_i = pix_i.pix_atoms[aid]
            start = 0 if pjx != pix else aid + 1

            # for all atom(s) in pix_j
            for bid in range(start, pix_j.patoms):
              # Set pointer to the second atom to test
              at_j = pix_j.pix_atoms[bid]

              # evaluate interatomic distance
              Dij = evaluate_distance(use_pbc, at_i, at_j)

              if Dij.length < cutoff_squared:
                # This is a bond!
                pass

      # store that pixel pix_i was tested
      pix_i.tested = True
```